

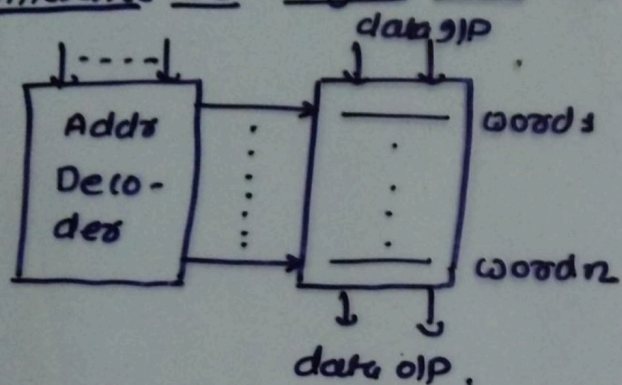
Associative mem →

①

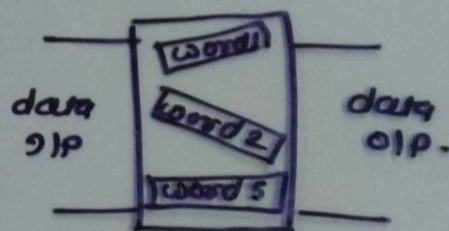
- Normally input is noisy or corrupted pattern & associative mem gives noise free pattern.

eg → ability to recognise the person either in person or from a photo even if appearance of a person is changed, which is relatively difficult by traditional comp algorithm.

* Difference betⁿ Digital mem & NN mem.



Digital mem



NN mem.

- ⇒ addr - addressable mem.
- ⇒ Has data inp & o/p lines.
- ⇒ Activation takes place when addr is decoded by addr decoder.
- ⇒ sequential search

- ⇒ Robust & fault tolerance ability.
- ⇒ Search is parallel.
- ⇒ Recall based on degree of similarity betⁿ input pattern & stored patterns.
- ⇒ Recall based on prior knowledge of its content.

Associative mem can be static or dynamic

once the inp is applied, in one feed forward pass, without any delay o/p is calculated

o/p is recalled as a result of inp, o/p feed back interactions.

- Neural mem can be implemented by using either feed forward n/w or recurrent n/w.

* performance parameters $\rightarrow$ measured by

(2)

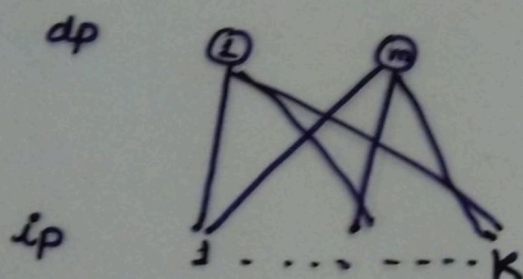
- (a) Information capacity
- (b) ability to retrieve correct info.

* disadv $\rightarrow$ (a) Capacity of NW may be restricted.

(b) Sometimes spurious patterns may be generated.

Hebb associative rule $\rightarrow$

$$T_p = \{(I_p, d_p) ; p = 1 \dots P\}$$



$$I_{pk} = [I_{p1} \dots I_{pn}] ; k = 1 \dots n$$

$$d_{pj} = [d_{p1} \dots d_{pm}] ; j = 1 \dots m$$

w_{jk} = weight from j^{th} neuron to k^{th} o/p.

Hebb's rule $\rightarrow$

$$w_{jk} = \sum_{p=1}^P d_{pj} \cdot I_{pk}$$

$$W = D \cdot I^T \rightarrow \begin{bmatrix} \text{matrix whose rows} \\ \text{are o/p vectors} \end{bmatrix}$$

$\downarrow$

$$\begin{bmatrix} \text{matrix whose col.} \\ \text{are o/p vectors.} \end{bmatrix}$$

Works in two phases $\rightarrow$

training $\rightarrow$ Develop wt matrix.

testing/Retrieval $\rightarrow y = \text{sgn}(w^t x)$ It is process of generation of o/p pattern when i/p pattern is presented to NW.

Associative mem.

(3)

- If incorrect or corrupted data is given as ip, o/p for this will correspond to one of stored patterns in mem.

eg → In response to incorrectly spelled word, human being retrieves associated correct word in mem.

many a times if a word is not spelled correctly, its meaning will be correctly retrieved by user.

- In case of human being info. is stored in the form of complex inter connections betⁿ neurons.

- ANN can play a role of associative memory. The data is stored in the wt matrix & wt matrix is used to generate o/p for corresponding ip.

- Association → Task of mapping patterns to patterns.

Hetero-association

- mapping of ip vectors to o/p vectors that range over diff. vector space.

eg → English to Spanish translation, code conversion.

lookup table.

Auto-association

- mapping of ip vectors to o/p vectors that range over same vector space.

eg → 1. spelling corrector maps possibly incorrectly spelled words to correctly spelled words in some language.

- 2. Any distorted & noisy ip patterns will be mapped to noise-free & unclustered o/p data.

- Patterns are stored in mem. in the form of connection wt vectors.

- no. of patterns = size of mem.

- Learning → Process of developing wt matrix.

storing desired pattern.

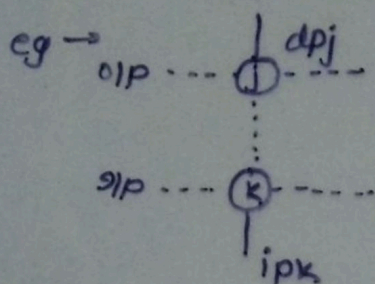
- Retrieval → Generation of o/p pattern when ip pattern is presented to the n/w.

1. HEURO-ASSOCIATIVE NN →

(4)

Principle → A specific cell rapidly assists in firing of another cell in the sys. Axon over the process develops synaptic connections with dendrites of other cell.

i.e. change in wt betn 2 nodes & correlation betn amplitudes of respective nodes.



$ip = ip_1, \dots, ip_k$, $dp = dp_1, \dots, dp_j$
values vary $\{-1, +1\}$ values vary $\{-1, +1\}$

$$\therefore \Delta w_{jk} \propto dp_j \cdot ip_k$$

this is Hebb's rule.

If all o/p & o/p patterns are available at once →

$$w_{jk} = \sum_{p=1}^P dp_j \cdot ip_k.$$

• The exact form of w_{jk} depends on node fun used & n/w used.

• methods of wt updation →

1. single step association or noniterative procedure or matrix associative mem. [MAM]
2. iterative procedure / least square procedure.
3. optimal linear associative mem. [OLAM]

① MAM →

Transform o/p vector $x^{(1)}$ by premultiplication with wt matrix
 $y = w \cdot x^{(1)}$

If desired o/p is bivalent then.

$$o/p = \text{sgn}(y).$$

• All the wts are updated simultaneously with Hebb's rule in single step.

$$w_{jk} = \frac{1}{P} \sum_{p=1}^P w_{pj} \cdot ip_k.$$

• For bivalent o/p we are interested in sign of each component of y . $\therefore ip$ can be omitted.

$$\therefore w_{jk} = \sum_{p=1}^P w_{pj} ip_k = \sum_{p=1}^P dp(ip)^T$$

$$= D \cdot I^T$$

$\therefore w = [\text{matrix whose columns are o/p vectors}] \times [\text{matrix whose rows are i/p vectors}]$

• Iterative Process of wt update → [Least Square Procedure].

• This method is used to derive wt matrix Ω that minimizes mean squared error.

• When o/p pattern ip is presented to n/w, resulting o/p - Ωip must be as close as possible to expected o/p dp .

$$\therefore E = \sum_{p=1}^P \|dp - \Omega ip\|^2$$

$$= \sum_{p=1}^P \left[(dp_1 - \sum_j \Omega_{1j} \cdot ip_j)^2 + \dots + (dp_m - \sum_j \Omega_{mj} \cdot ip_j)^2 \right]$$

for min error $\frac{\partial E}{\partial \Omega_{jl}} = 0$

$$\begin{cases} ip_l = \{-1, 1\} \text{ for } l = 1 \dots n \\ dp_j = \{-1, 1\} \text{ for } j = 1 \dots m. \\ n \text{ dimensional vector.} \end{cases}$$

$$\therefore \frac{\partial E}{\partial \Omega_{jl}} = -2 \sum_{p=1}^P (dp_j - \sum_{k=1}^n \Omega_{jk} \cdot ip_k) \cdot ip_l = 0$$

$$\therefore \sum_{p=1}^P dp_j ip_l = \sum_{p=1}^P \sum_{k=1}^n \Omega_{jk} \cdot ip_k \cdot ip_l$$

$$= \sum_{k=1}^n \Omega_{jk} \left(\sum_{p=1}^P ip_k \cdot ip_l \right)$$

$$\therefore D[J^t] = \Omega [J \cdot J^t]$$

$$\Omega = D[J^t] \cdot [J \cdot J^t]^{-1}$$

where

D = matrix whose column corresponds to desired o/p pattern

J = matrix whose column " " to ip pattern.

• Optimal Linear associative mem. [OLAM].

• Every matrix does not have inverse. But every matrix has pseudo inverse i.e A^* .

It satisfies following conditions.

$$A \cdot A^* \cdot A = A$$

$$\& \Omega = D[J]^*$$

$$A^* \cdot A \cdot A^* = A^*$$

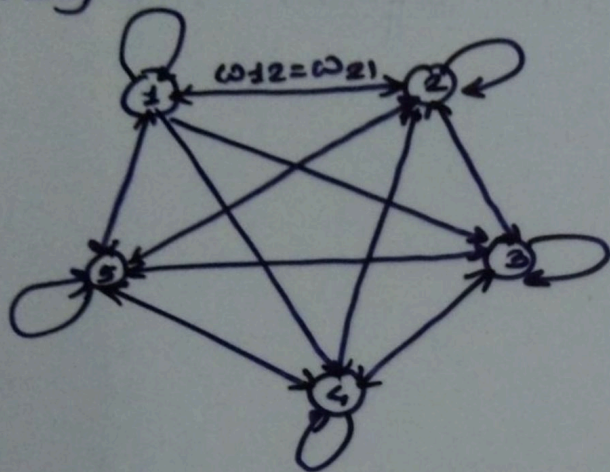
$$A^* \cdot A = (A^* \cdot A)^T$$

$$A \cdot A^* = (A \cdot A^*)^T$$

⑥

HOPFIELD NN →

- Normally used for auto association.
- Fully connected NN & wts are updated by Hebbian principle.



- Node values are iteratively updated based on local computation principle.
- No lengthy procedure required to determine wt matrix.
- System has to undergo large no of transitions before converging to stable state.
- But still no of iterations req are smaller than that of feed forward NN.

• Hopfield n/w $\rightarrow$ mostly used for auto-association.

• Node values are updated iteratively.

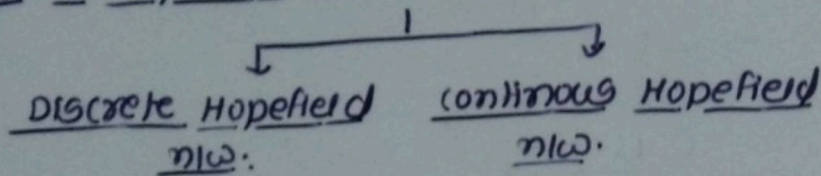
- New state of each node depends only on its net-acted o/p at any given time.

- The n/w is fully connected. & wts are updated by Hebbian principle.

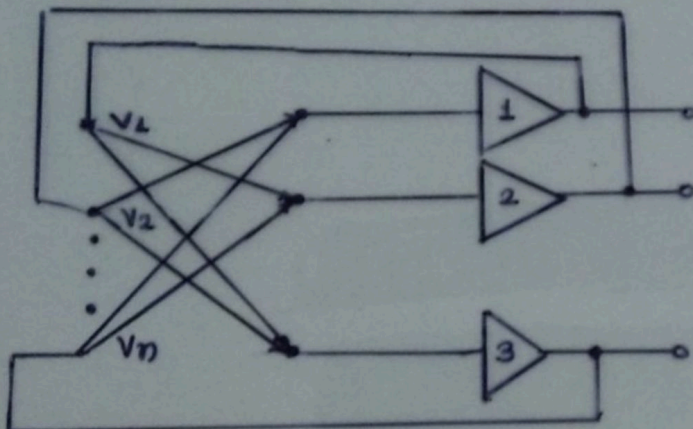
- Before converging to stable state sys has to undergo large no. of state transitions.

- But still no. of iterations req. are less than that feedforward n/w.

• Two types of Hopfield n/w's $\rightarrow$



1. Discrete Hopfield n/w $\rightarrow$



• The nodes are assumed to be bivalent.

• Each node uses step activation function.

- o/p & o/p patterns are discrete vectors & values can be either 0 or 1.

- Node o/p may change when input vector is presented, till state of n/w converges to an attractor.

- As computations proceed time variations that range over a natural no., it is called as discrete time Hopfield n/w.

Let $\{I_1, \dots, I_p\}$ is set of n dimensional pattern to be stored.

$v_{pj}(t)$ = At time t , o/p generated by j th node when p th pattern activates the n/w.

w_{kj} = connection wt from j th node to k th node.

I_{pk} = External o/p to k th node of n/w for p th o/p vector.

• wt updation can be synchronous or asynchronous →

1. Asynchronous →

- At every time instant, one node old is updated.
- selection of node for update may be at random.
- often selection of nodes is cyclic.
- select node to be updated.

$$u_{pk}(t+1) = u_{pk}(t) \quad \text{if } l \neq k$$

$$= \text{sgn} \left(\sum_{j=1}^n \omega_{kj} u_{pj}(t) + i_{pk} \right) \quad \text{if } l=k$$

- It gives guarantee of stabilization.

2. Synchronous model →

- At every instant of time all nodes are updated simultaneously.

$$u_{pk}(t+1) = \text{sgn} \left(\sum_{j=1}^n \omega_{kj} x_{pj}(t) + i_{pk} \right).$$

- It always does not guarantee convergence.

Applications → To retrieve stored pattern when corrupted version of stored pattern is presented.

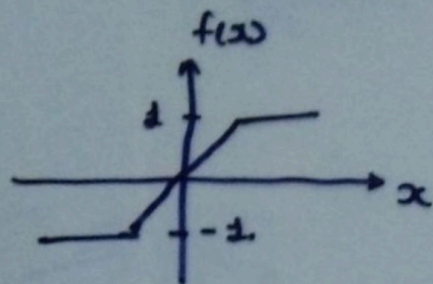
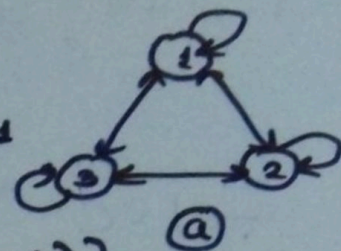
- Also used to complete pattern when parts of pattern are missing.

- sometimes [w/ new o/p vector] may result into o/p pattern that is not stored in mem. (9)
- this type of o/p is spontaneous o/p.
- Disadv → when mem is stimulated by o/p patterns that differs even slightly from o/p patterns used to build W often spontaneous o/p will result i.e. they have low error correction capability.

Brain state in box →

(10)

- It is used for autoassociation & can be extended for Heteroassociation with two or more layers of nodes.
- It is fully connected n/w with as many nodes as dimensionality of obj space.
- All nodes are updated simultaneously.
- nodes take value in continuous range from -1 to 1
- Node fun = ramp.



$$f(\text{net}) = \min(1, \max(-1, \text{net}))$$

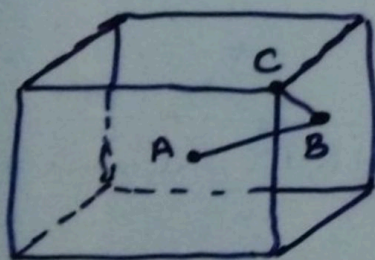
- It is bounded, continuous & piecewise linear.

State of node changes according to

$$x_i(t+1) = f\left(\sum_{j=1}^n w_{ij} \cdot x_j(t)\right)$$

w_{ii} = self excitation parameter = 1

- For this, initial activation is steadily amplified by +ve fb & node activation saturates at +1 or -1. Each node activation belongs to close interval $[-1, 1]$ so that state of n/w always remains inside n dimensional box. ∴ It is called BSB



- Due to ramp fun, state of n/w steadily moves from an arbitrary pt inside the box towards one side of the box & then crawls along the side of box to reach a vertex of box.

- ④ shows n/w with three nodes. Each possible obj pattern corresponds to vertex of cube.

When supplied with initial stimuli A, state of n/w may be modified till B reaches, a pt on face of cube. Further modification may lead to state C.

- connection nodes are Hebbian & represents co-relation betn nodes. connection wts can be updated by iterative / non iterative computations.

① Non Iterative →

Assumes fixed set of training patterns with wt associated with connection from j^{th} node to i^{th} node, obtained using eqn

$$w_{ij} = \frac{1}{P} \sum_{p=1}^P i_{p,i} \cdot i_{p,j} \quad \forall j \& i.$$

$i_{p,i}$ = i^{th} component of p^{th} pattern

② Iterative procedure →

Training patterns are repeatedly presented to the n/w & wts are modified as,

$$\Delta w_{ij} = \eta i_{p,j} (i_{p,i} - \sum_k w_{ik} i_{p,k}) \quad \text{for } p=1, \dots, P$$

continue till $\Delta w_{ij} = 0$

i.e. when desired pattern = actual pattern

$$i_{p,i} = \sum_k w_{ik} i_{p,k}$$

the training is complete

- During ~~training~~ application phase wts remain same as

$$w_{ij} = \frac{1}{P} \sum_{p=1}^P i_{p,i} \cdot i_{p,j}$$

Least square procedure →

(12)

① First way of finding wt matrix — by using Hebb's rule

$$W = D I^T$$

② other way of generating wt matrix is by minimizing error bwn desired o/p & actual o/p.

$$E = \sum_{p=1}^P (d_p - x_p)^2$$

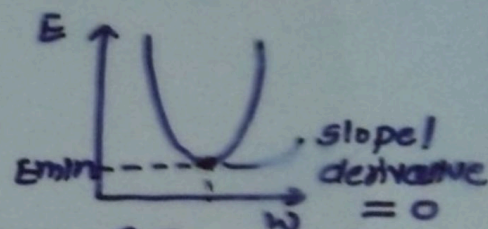
d_p = desired o/p
 x_p = actual o/p
 $= w \cdot i_p = \Omega \cdot i_p$

$$i_{p\ell} = \begin{bmatrix} i_{\ell 1} & \dots & i_{\ell n} \\ \vdots & & \vdots \\ i_{p1} & \dots & i_{pn} \end{bmatrix} \quad d_{pj} = \begin{bmatrix} d_{p1} \\ \vdots \\ d_{pm} \end{bmatrix} \quad \Omega_{mj} = \begin{bmatrix} \Omega_{11} & \dots & \Omega_{1j} \\ \vdots & & \vdots \\ \Omega_{m1} & \dots & \Omega_{mj} \end{bmatrix}$$

The wt matrix Ω should be chosen to minimize mean square error.

$$E = \sum_{p=1}^P (d_p - \Omega \cdot i_p)^2 \quad \text{--- (A)}$$

$$= \sum_{p=1}^P (d_{p1} - \sum \Omega_{1j} \cdot i_{pj})^2 + \dots + (d_{pm} - \sum \Omega_{mj} \cdot i_{pj})^2$$



- error function is quadratic in nature & at min of error function its derivative is zero.

∴ To find out wt at min error differentiate eqn (A) w.r.t $\Omega_{j\ell}$

$$\frac{\partial E}{\partial \Omega_{j\ell}} = +2 \left(\sum_{p=1}^P (d_{pj} - \sum_{k=1}^n \Omega_{jk} \cdot i_{pk}) \right) (-i_{p\ell}) = 0$$

$$\begin{aligned} \therefore \sum_{p=1}^P d_{pj} \cdot i_{p\ell} &= \sum_{p=1}^P \sum_{k=1}^n \Omega_{jk} \cdot i_{pk} \cdot i_{p\ell} \\ &= \sum_{k=1}^n \Omega_{jk} \sum_{p=1}^P i_{pk} \cdot i_{p\ell} \end{aligned}$$

in matrix form

$$\sum_{p=1}^P d_p \cdot i_{p\ell} = \sum_{p=1}^P d_p \cdot i_p^T = \Omega \sum_{p=1}^P i_p \cdot i_p^T$$

$$\therefore \Omega = \left(\sum_{p=1}^P d_p \cdot i_p^T \right) \left(\sum_{p=1}^P i_p \cdot i_p^T \right)^{-1} = (O \cdot I^T) (I \cdot I^T)^{-1}$$

For BSB $\rightarrow$

$i_{31} \ i_{32} \ i_{33}$
 $(1, 1, 1)$
 $(-1, -1, -1)$
 $(1, -1, -1)$
 $i_{11} \ i_{12} \ i_{13}$

$$\omega_{ij} = \frac{1}{P} \sum_{p=1}^P i p_L \cdot i p_j$$

$$\omega_{12} = \frac{1}{3} \sum_{p=1}^3 i p_1 \cdot i p_2$$

$$\omega_{12} = \frac{1}{3} (i_{11} \cdot i_{12} + i_{21} \cdot i_{22} + i_{31} \cdot i_{32})$$

For BAM $\rightarrow$

$(1, 1, -1, -1) \rightarrow (1, 1)$
 $(1, 1, 1, 1) \rightarrow (1, -1)$
 $(-1, -1, 1, 1) \rightarrow (-1, 1)$

$$\omega_{ij} = \frac{1}{P} \sum_{p=1}^P i p_j \cdot d p_L$$

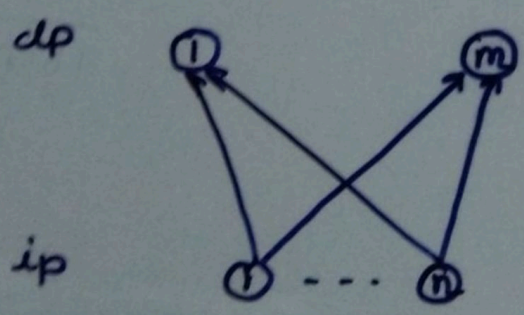
$$\omega_{11} = \frac{1}{3} \sum_{p=1}^3 i p_1 \cdot d p_1$$

$$\omega_{11} = \frac{1}{3} (i_{11} d_{11} + i_{21} \cdot d_{21} + i_{31} \cdot d_{31})$$

MAM $\rightarrow$

$$\omega_{ij} = \sum_{p=1}^P i p_j \cdot d p_L$$

$$= D \cdot I^T$$



$$i p_j = [i p_1 \dots i p_n]$$

$$d p_L = [d p_1 \dots d p_m]$$

$$j = 1 \dots n$$

$$L = 1 \dots m$$